

VISÃO — Aprofundamento Técnico

Provas vivas do seu repositório: código, medições reais e derivações

Preparado por ÍRIS · Agosto de 2026 · Projeto VISÃO / Juan Guerra

Este material **aprofunda** o curso base. Aqui não há slogan: cada afirmação é ancorada em (a) código que existe em `/home/juan/projeto-visao`, (b) JSONs de medição reais, ou (c) uma derivação que você pode repetir. Nos blocos «Prova viva» eu indico exatamente o que rodei nesta sessão. Tudo verificável por você.

1. 1. O solver fundido do `cfc.py`: o que ele realmente é (e o que não é)

No `visao/core/cfc.py` a dinâmica **não** é integrada por Euler a cada passo. Ela usa um **solver fundido** (Euler semi-implícito) que é incondicionalmente estável para $dt > 0$, $\tau > 0$, $f \geq 0$. Parta da EDO da LTC:

$$\frac{dx}{dt} = -\left[\frac{1}{\tau} + f(x, I)\right]x + f(x, I)A \quad (1)$$

Nota Correção de nomenclatura (importante). O que está em `cfc.py` é uma **LTC com solver fundido** (discretização Euler semi-implícita da EDO), não o CfC fechado de Hasani et al. 2022. O CfC verdadeiro aproxima a trajetória inteira por uma **interpolação sigmoideal** entre duas sub-redes ponderada por $\sigma(-ft)$, sem iterar passo a passo — pula de $t = 0$ a qualquer t . O repositório oficial da Hasani tem inclusive uma **flag** para rodar a LTC com solver de EDO semi-implícito em vez do CfC completo: é exatamente o esquema deste arquivo. Nome correto: «LTC com solver fundido» ou «ODE-LTC». A prova de estabilidade abaixo continua válida — só muda o rótulo. MAT-22 vai dar o vocabulário formal (rigidez de EDO, métodos implícitos vs explícitos): tratar o termo em x de forma implícita é o mesmo truque de décadas em simulação de reação química e circuitos.

Derivação passo a passo **Passo 1 — discretizar com Euler explícito/implícito fundido.** Escreva $\alpha = \frac{1}{\tau} + f$. Então $\dot{x} = -\alpha x + fA$. O esquema de Euler de um passo, tratado de forma que o termo em x apareça dos dois lados:

$$x_{t+1} = x_t + dt(-\alpha x_{t+1} + fA) \quad (2)$$

Note que x_{t+1} aparece nos dois lados (implícito no vazamento) — isto é o que garante estabilidade incondicional.

Passo 2 — isolar x_{t+1} .

$$x_{t+1} + dt\alpha x_{t+1} = x_t + dtfA \quad (3)$$

$$x_{t+1}(1 + dt\alpha) = x_t + dtfA \quad (4)$$

$$x_{t+1} = \frac{x_t + dtfA}{1 + dt(\frac{1}{\tau} + f)} \quad (5)$$

É exatamente a linha 19 do `cfc.py`: `num = x + dt*f*A ; den = 1 + dt*(1/tau + f) ; return num/den`. O denominador é sempre > 1 (pois $dt, \tau, f > 0$), logo **nunca** divide por zero e o estado não explode. Este é o motivo técnico da estabilidade incondicional do solver (não confunda com «Closed-form» no sentido do CfC de Hasani — aqui é passo fechado de integração, não função fechada de t): a solução de um passo é analítica, não numérica.

Código real

```
# visao/core/cfc.py (trecho _step_raw)
def _step_raw(self, x, u):
    fx = self._f_raw(x, u)          # sigmoid(W_in@u + W_rec@x + b)
    num = x + self.dt * fx * self.A
    den = 1.0 + self.dt * (1.0/self.tau + fx)
    return num / den, fx
```

A não-linearidade f é a sigmoid: $f = \sigma(W_{in}u + W_{rec}x + b) \in (0, 1)$. Como $0 \leq f \leq 1$, o termo $1 + dt(\frac{1}{\tau} + f) \geq 1 + d\frac{1}{\tau} > 1$ — denominador protegido.

Prova viva — rodei isto Rodei a célula de verdade (seed=7, n=96, dt=0.1) e medi a **liquidez** — a faixa de $\tau_{ef} = \frac{\tau}{1+\tau}$ sob entrada calma vs agitada:

calma : tau_ef [0.334, 1.404] media 0.762
 agitada: tau_ef [0.306, 3.423] media 0.814
 razão faixa (agit/calma) ≈ 2.9 –3.3x

O projeto reporta $3,282\times$ (razão de faixa agitada/calma). Meu número bateu a mesma ordem — a diferença vem da semente e do comprimento da sequência. **A célula responde ao mundo**: sob sinal forte, $f \rightarrow 1$ e $\tau_{ef} \rightarrow \frac{\tau}{1+\tau}$ (mais curto); sob sinal fraco, $f \rightarrow 0$ e $\tau_{ef} \rightarrow \tau$ (mais longo). Memória multiescala emergente.

2. 2. Por que o esquecimento catastrófico some: a ablação matemática

O `prototype/results_continual.json` tem os números crus de 5 seeds. A redução de 97,2% não é «média de resumo» — é a diferença entre duas distribuições de **forgetting** medido por tarefa.

A métrica de esquecimento de A após treinar B é:

«forgetting»(A) = $E_A(\text{«após treinar B»}) - E_A(\text{«logo após treinar A»})$

Ou seja: quanto piorou a tarefa A. No JSON, para a célula **plástica** (seed 0): «mean forgetting» = 0,0049; para o controlador **naive**: «mean forgetting» = 0,1968. Redução: $1 - 0, \frac{0049}{0}, 1968 \approx 97,5\%$ (o 97,2% usa as médias agregadas do summary).

Nota Detalhe que os resumos escondem: no seed 2 e 3 da célula plástica, o forgetting de A foi **negativo** (-0,0002, -0,0009). Isso significa que treinar B **melhorou** A — um efeito de consolidação, não só de não-esquecer. O naive nunca tem isso: seus forgettings de A são todos positivos e enormes (0,39 a 0,88).

A ablação $2\times 2\times 2$ (BACKLOG 1.9) isola **quem** cause isto. A descoberta honesta:

- Componente principal: surpresa (Δ forgetting $\approx -0,262$ isolado).
- Oja isolado: $-0,092$ (reduz, mas pouco).
- Consolidação **sozinha**: $+0,178$ — aumenta o esquecimento. Só ajuda em combinação, como freio anti-divergência.
- Surpresa ligada **sem** consolidação diverge (pesos explodem) — reportado como «divergiu», não como vitória.

Nota A história que o curso abre está errada — e a ablação já dizia isso. Os capítulos 1-2 do curso base vendem «peso local esquece menos porque só quem sabe de A mexe no peso de A». Mas a ablação mostra o oposto: Oja isolado (a parte **genuinamente local**) reduz só $-0,092$; surpresa isolada é $-0,262$ (o componente principal); consolidação sozinha **piora** ($+0,178$). O que protege memória não é **localidade** — é **quando** você atualiza (só sob surpresa), não **quem pode** atualizar. Descoberta legítima, só de outra família da que abre o curso.

Duas conexões úteis: (1) Oja (1982) não é só «Hebb com freio» — é **PCA online**: o vetor de peso converge para o autovetor principal da covariância da entrada (o mesmo motor do PCA em streaming). (2) Existe uma abordagem inteiramente **global** — **Elastic Weight Consolidation** (Kirkpatrick et al., PNAS 2017) — que penaliza mexer em pesos importantes para tarefas antigas via informação de Fisher, e funciona bem apesar de ser tão global quanto backprop. Reforça o que a ablação sugere: o eixo que importa é proteger atualização por **importância** (sinalizada localmente por surpresa, ou globalmente por Fisher), não onde o peso mora. Experimento barato pra isolar isso: um EWC guiado por surpresa em vez de Fisher teria a mesma cara de curva que vocês mediram?

Para o ITA No ITA isto é **análise de variância** e **design de experimentos**: grade fatorial $2\times 2\times 2$ (3 fatores, 2 níveis = 8 células, 5 replicatas). Você vai ver exatamente este tipo de isolamento de efeito em estatística experimental. O VISÃO já faz.

3. 3. Estabilidade: espectro, Lyapunov e a borda do caos

O `visao/analysis/results_stability.json` é o laudo real (n=96, dt=0,1, T=600, entrada agitada):

```
"spectral_radius_Wrec": 0.7577,
"max_jacobian_spectral_radius": 0.9754,
"lyapunov_exponent": -0.7484,
"state_norm_final": 3.726,
```

```
"state_dynamics_std": 0.306,
"verdict": "SUB-CAOTICO (LE<0) e NAO-MORTO"
```

Derivação passo a passo O que estes números significam.

1. **Raio espectral** $\rho(W_{rec}) = 0,7577 < 1$. Para um Echo State Network, o critério de «reservatório bem comportado» é $\rho < 1$ (estado não explode). Passou.
2. **Maior raio espectral do jacobiano local** $0,9754 < 1$. O jacobiano de um passo é $J = I - dt(\frac{1}{\tau} + f') + dt f' W_{rec}$ (aprox.). Seu maior autovalor em módulo < 1 significa que o mapa é **contrativo** — perturbações encolhem a cada passo. Passou.
3. **Expoente de Lyapunov** $\lambda = -0,7484 < 0$. Trajetórias vizinhas **convergem**. O sistema é sub-caótico: tem memória estável (não esquece) mas não diverge.
4. **Norma final** 3,726 e desvio 0,306 (não morto): o estado responde à entrada, não colapsa a zero. Vivo e estável.

Prova viva — rodei isto A medição correta de Lyapunov (algoritmo de Benettin, 1980). Tentei medir «na mão» perturbando o estado inicial em 0,01 — deu — inf no log. Isso **não** é limitação do método: é o sintoma clássico de medir Lyapunov com perturbação única sem renormalizar. Num sistema fortemente contrativo ($\lambda \approx -0,75$), qualquer δx inicial encolhe exponencialmente e estoura o underflow de ponto flutuante antes de $T = 600$. O remédio padrão (Benettin) é: evolua a perturbação 1 passo, meça $\log(\|\delta x'\|/\|\delta x\|)$, **renormalize** δx de volta a ϵ , repita. $\lambda \approx$ média dos logs. Rodei isto:

```
Benettin: 600 passos válidos
lambda (por passo)      = -0,0788
lambda (por unidade temp) = -0,7876
razão de contração média por passo = 0,9243
```

Duas estimativas independentes — espectro de jacobiano ($-0,7484$) e Benettin ($-0,7876$) — convergem no mesmo sinal (sub-caótico, contrativo) e na mesma ordem de grandeza. É assim que se confirma um número de estabilidade de verdade.

Nota «Borda do caos» — ajuste de vocabulário. O critério $\rho(W_{rec}) < 1$ que usamos é literalmente a **echo state property** de Jaeger (ESN, 2001) — reservoir computing «puro», não LTC. E «líquido» tem origem paralela: as **Liquid State Machines** de Maass, Natschläger & Markram (2002) se chamam assim porque o estado do reservatório é como a superfície de um líquido perturbada por uma pedra (o pulso de entrada) — nada a ver com a «constante de tempo líquida» da Hasani. Três linhagens (LSM, ESN, LTC) convergem no mesmo vocabulário por boas razões matemáticas, mas **não** são a mesma família. E o número avisa: $-0,7484$ não está «na borda» — está razoavelmente fundo na zona estável. Para um protótipo é a escolha certa, mas na literatura de reservatório (Bertschinger & Natschläger, 2004) a capacidade computacional de um reservatório é **maximizada** perto de $\lambda \approx 0^-$. Vale, mais adiante, tunar para λ mais perto de zero e comparar a memória dos dois regimes.

4. 4. FedAvg: a média que converge (e a prova de $< 5\%$)

visao/swarm/node.py implementa treino local e o coordenador faz a média ponderada. A equação (Fase 2.1):

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} \cdot w_{t+1}^{(k)}, \quad n = \sum n_k \quad (6)$$

A prova de convergência do teste (BACKLOG 2.1): dois processos, 200 amostras (shards 100/100), lr=0,01, K=5, 20 rodadas. O modelo distribuído convergiu para o **mesmo** estado que single-node com erro relativo $< 5\%$.

Derivação passo a passo Por que a média de pesos faz sentido (intuição de gradiente). Cada nó faz descida de gradiente local: $w_k \rightarrow w_k - \eta \nabla L_k$. Se todos partem de w_0 e dão um passo ($K = 1$), a média dos pesos é:

$$\sum_k \frac{n_k}{n} (w_0 - \eta \nabla L_k) = w_0 - \eta \sum_k \frac{n_k}{n} \nabla L_k \quad (7)$$

Mas $\sum_k \frac{n_k}{n} \nabla L_k = \nabla \left(\sum_k n_k \frac{L_k}{n} \right) = \nabla L_{pool}$ — o gradiente da perda **poolsada** (todos os dados juntos). Portanto, com $K = 1$, a média de uma passada de cada nó = **um passo de gradiente descendente sobre o dataset completo**. FedAvg é descida de gradiente distribuída. Daí convergir para o mesmo ótimo do single-node.

Nota A derivação acima é exata só com $K = 1$. O teste de vocês usa $K = 5$ passos locais por rodada. Com $K > 1$ isso deixa de ser exatamente SGD centralizado — funciona bem quando os shards são parecidos ou K é pequeno, e degrada sob **heterogeneidade** (os dados dos nós diferem). É o fenômeno de **client drift**, estudado após o FedAvg original (McMahan et al. 2017) por trabalhos como SCAFFOLD (Karimireddy et al. 2020). Shards 100/100 idênticos não revelam isso ainda — mas é exatamente o que vai aparecer na Fase 2.2 com voluntários de dados e hardware heterogêneos. Vale preparar correção de drift (SCAFFOLD, FedProx) antes de escalar.

Código real

```
# visao/swarm/node.py (trecho _grad - regressão linear como prova vertical)
def _grad(params, X, Y):
    err = (X @ params["w"] + params["b"]) - Y
    return {"w": (2/n)*(X.T @ err), "b": (2/n)*err.sum(...)}
# o CfCCell pluga depois como qualquer dict de pesos (arquitetura-agnóstico)
```

5. 5. Governança: como o código impõe R1–R5

visao/governance/containment.py é o único arquivo auditável. Ele **levanta exceções nomeadas** para cada violação — «tudo falha fechado» não é retórica, é **raise**.

Código real

```
# visao/governance/containment.py (excerto)
class ContainmentViolation(Exception): pass
class QuorumDenied(ContainmentViolation): # R1
class LifetimeExpired(ContainmentViolation): # R2
class BoundaryBreached(ContainmentViolation): # R3
class ThrottleExceeded(ContainmentViolation): # R4
class TelemetryCorrupted(ContainmentViolation): # R5

def _h(data: bytes) -> str:
    return hashlib.sha256(data).hexdigest() # selo R3
```

O comentário no topo do arquivo é explícito: «ESTE ARQUIVO ESTÁ FORA DO ESPAÇO DE MUTAÇÃO (regra R3) [...] O hash deste arquivo é verificado a cada ciclo contra manifest.json. Divergência = parada imediata.»

A Fase 6.3 embute esse hash no bundle do cérebro e o recalcula no `load()`:

Derivação passo a passo R3 self-verify (por que funciona). Seja $H = \text{hash}(C)$ o selo de `containment.py` no manifesto. O bundle carrega H e o código atual C . No `load`:

«se» $H \neq \text{hash}(C)$ «então» «raise `BoundaryBreached`»

Como H vem do manifesto selado (fora do espaço de mutação), e C é o código que está rodando, qualquer adulteração a quente de `containment.py` faz $\text{hash}(C) \neq H \rightarrow$ parada. O sistema **não consegue** desligar sua própria trava, porque a trava é verificada por um valor que ele não pode alterar.

Nota R4 em números: o custo de criar c filhos cresce como $2^{8+1,6c}$ tentativas de hash. Para $c = 5$ filhos, isso é $2^{8+8} = 2^{16} = 65536$ hashes — já um freio computacional real. Para $c = 10$, $2^{8+16} = 2^{24} \approx 16$ milhões de hashes. A replicação exponencial vira economicamente inviável muito rápido. Combinado com R1 (quórum humano), nenhum «fork» em cascata acontece sem assinaturas.

6. 6. Contagem de parâmetros: a vantagem paramétrica real

visao/bench/results_param_count.json é determinístico (não depende de treino):

```
"parametros_treinaveis": {"CfC": 5002, "LSTM": 67850, "GRU": 50816},
"razao_LSTM_sobre_CfC": 13.56,
"razao_GRU_sobre_CfC": 10.16
```

Derivação passo a passo Por que o CfC tem tão poucos parâmetros. O CfC numa tarefa 1-D (psMNIST, $n_{in}=1$) com $h=64$:

- $W_{in} : 64 \times 1 = 64$
- $W_{rec} : 64 \times 64 = 4096$ (menos a máscara de esparsidade, mas conta-se o total)
- $b : 64$
- $A : 64$ (potencial de reversão — um vetor, não uma matriz)
- $\tau : 64$

Total $\approx 64 + 4096 + 64 + 64 + 64 = 4352$, mais o readout ≈ 5002 .

Já a LSTM ($h=128$) tem 4 portas (input/forget/cell/output), cada uma $128 \times 128 + 128 \times 1 + \text{bias}$: $4 \times (16384 + 128 + 128) \approx 66688$, mais readout ≈ 67850 . O CfC faz **um** passe de acoplamento não-linear em vez de quatro; daí a diferença de uma ordem de grandeza.

Nota Honestidade: no benchmark psMNIST 60k (Colab T4), o CfC ficou **12,1 pp** atrás do LSTM em acurácia (0,6939 vs 0,8149). A vantagem do CfC é **paramétrica** ($13,56\times$ menos), não de acurácia bruta. O projeto escreveu isto sem maquiagem: «critério de sucesso não atingido; critério de falha não dispara» (gap $12,1\% < 15\%$). O GRU em 60k não foi medido (queda de sessão Colab) — e o número **não foi inventado**.

7. 7. Exercícios de aprofundamento (com respostas)

Exercício No solver fundido, prove que $x_{t+1} \in [0, A]$ se $x_t \in [0, A]$ e $f \in [0, 1]$, $A > 0$. (Dica: é uma média convexa.)

Exercício Mostre que o denominador $1 + dt(\frac{1}{\tau} + f) \geq 1 + d\frac{t}{\tau}$ e por que isto implica estabilidade incondicional (independe de dt).

Exercício Usando $\tau = 1$ e $f = \sigma(z)$ com z variando de $-\inf$ a $+\inf$, qual o intervalo de τ_{ef} ? O que isto diz sobre o «quanto» a célula desacelera?

Exercício No FedAvg, se $n_1 = 3n_2$ (nó 1 tem $3\times$ mais dados), escreva w_{t+1} em função das duas cópias. Por que isto é justo (evita viés do nó pequeno)?

Exercício Explique por que a ablação mostrou que consolidação **sozinha** aumenta o esquecimento, mas em combinação ajuda. (Dica: papel anti-divergência do freio de consolidação.)

7.1. 7.1 Respostas

1. $x_{t+1} = \frac{x_t + dt f A}{1 + dt(\frac{1}{\tau} + f)}$. Como $x_t, A, f, dt, \tau > 0$, numerador e denominador são positivos. Reescreva $x_{t+1} = \theta x_t + (1 - \theta)A$ com $\theta = \frac{1}{1 + dt(\frac{1}{\tau} + f)} \in (0, 1)$: é média convexa de x_t e A , logo está em $[\min(x_t, A), \max(x_t, A)] \subset [0, A]$. Preservado.
2. $1 + dt(\frac{1}{\tau} + f) \geq 1 + d\frac{t}{\tau} > 1$ para qualquer $dt > 0$ (pois $\frac{1}{\tau} > 0$). Não há divisão por zero nem amplificação; o passo é sempre estável, independente do tamanho de dt . É por isso que «incondicionalmente estável».
3. $f = \sigma(z) \in (0, 1)$: $\tau_{ef} = \frac{1}{1+f}$ (com $\tau = 1$) varia de $\frac{1}{1+0} = 1$ (sinal nulo) a $\frac{1}{1+1} = 0,5$ (sinal saturado). A célula **no máximo** reduz à metade seu tempo efetivo — «desacelera» até $2\times$, não mais. A faixa de memória é limitada mas real.
4. $w_{t+1} = \frac{3}{4}w_{t+1}^{(1)} + \frac{1}{4}w_{t+1}^{(2)}$. O nó com mais dados pesa mais $\rightarrow$ a média não é dominada pelo nó pequeno; é a média ponderada correta pela contribuição de dados (igualdade de variância amostral).
5. Consolidação **sozinha trava** os pesos (sem surpresa que os atualize), mas o portão de surpresa ainda os perturba $\rightarrow$ deriva sem correção $\rightarrow$ esquecimento sobe. Em combinação, a consolidação **freia** a deriva causada pela surpresa (anti-divergência), permitindo aprender sem explodir. Daí ajudar só junto.

Fontes vivas consultadas: visao/core/cfc.py · visao/governance/containment.py · visao/swarm/node.py · prototype/results_continual.json · visao/analysis/results_stability.json · visao/bench/results_param_count.json · BACKLOG.json. Métricas de liquidez/rollout medidas pela ÍRIS nesta sessão (seed=7, n=96). Preparado por ÍRIS · Projeto VISÃO / Juan Guerra · Agosto de 2026.